

lecture explanation

Absolutely — here are clean, exam-friendly notes from the transcript, with the important Docker commands and concepts separated out.

Docker & Docker Hub — Day 2 Notes

1. What was covered

The lecture covered:

- Installing Docker
 - Verifying Docker installation
 - Understanding Docker Hub and Docker Registry
 - Understanding repositories, images, and tags
 - Pulling an image from Docker Hub
 - Creating a `Dockerfile`
 - Building a custom Docker image
 - Understanding Docker image layers
 - Running a container
 - Listing running/stopped containers
 - Tagging an image
 - Pushing an image to Docker Hub
 - Docker authentication
 - Useful Docker commands
-

2. Installing Docker

Docker Desktop is available for:

- macOS
- Windows
- Linux

Recommended system resources

Docker Desktop requires around **4 GB RAM**, but the lecture recommends:

- **8–12 GB RAM**
- **4 CPUs**

This is because your system also needs resources for your browser, IDE, applications, etc.

Alternative: Cloud VM

If your laptop isn't powerful enough, you can create an Ubuntu VM on a cloud provider such as:

- Azure
- Google Cloud

Then install **Docker Engine** on the Ubuntu VM.

Docker Desktop is convenient for a personal laptop, while Docker Engine on an Ubuntu cloud VM is also sufficient for learning.

3. Verify Docker Installation

After installation:

```
docker --version
```

or:

```
docker version
```

Difference

```
docker --version
```

- Gives a short version output.

```
docker version
```

- Gives detailed information about Docker components and versions.

4. Docker Hub

Docker Hub is a public Docker registry commonly used to store and distribute Docker images.

You can create a Docker Hub account and use the same credentials to sign in to Docker Desktop.

Docker Registry → Repository → Image

Think of the structure like this:

```
Docker Registry
  ↓
Repositories
  ↓
Images / Versions
```

For example:

```
Docker Hub
  ↓
ubuntu repository
  ↓
ubuntu:latest
ubuntu:22.04
ubuntu:24.04
...
```

Important terminology

Term	Meaning
Registry	Stores Docker repositories
Repository	Collection of related images
Image	Template used to create containers
Tag	Identifier/version of an image

5. Docker Image Naming

A Docker image can be represented as:

```
registry/username/image:tag
```

Example:

```
docker.io/cloudwithvaros/my-first-image:v1
```

Breakdown:

```
docker.io      → Registry
cloudwithvaros → Username / namespace
my-first-image → Image name
v1             → Tag
```

For official Docker Hub images, some parts can be omitted.

For example:

```
docker pull ubuntu
```

Docker interprets this approximately as:

```
docker.io/library/ubuntu:latest
```

Defaults

When you run:

```
docker pull ubuntu
```

Docker assumes:

```
Registry = docker.io  
Namespace = library  
Image = ubuntu  
Tag = latest
```

6. docker pull

Used to download an image from a registry.

```
docker pull ubuntu
```

This pulls the latest Ubuntu image.

To pull a specific version:

```
docker pull ubuntu:<tag>
```

Example:

```
docker pull ubuntu:24.04
```

General syntax

```
docker pull [registry/]repository/image[:tag]
```

7. docker help

To see available Docker commands:

```
docker --help
```

Docker provides commonly used commands in the help output.

Important commands from this lecture:

```
docker pull
docker push
docker images
docker build
docker run
docker ps
docker version
docker info
```

8. List Docker Images

Use:

```
docker images
```

This displays locally available images.

Typical output contains:

```
REPOSITORY
TAG
IMAGE ID
CREATED
SIZE
```

Example:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
ubuntu	latest	abc123	5 weeks ago	101MB

Important

ubuntu is the **repository**.

latest is the **tag**.

Together:

```
ubuntu:latest
```

identify a particular image.

9. Dockerfile

A **Dockerfile** is a text file containing instructions used to build a Docker image.

Example:

```
FROM ubuntu
CMD ["echo", "Hello Docker"]
```

FROM

```
FROM ubuntu
```

Specifies the **base image**.

Here:

```
Base image = Ubuntu
```

CMD

```
CMD ["echo", "Hello Docker"]
```

Specifies the default command that runs when a container starts.

10. Build a Docker Image

Docker uses the `docker build` command to create an image from a Dockerfile.

Example:

```
docker build -t my-first-image .
```

Breaking it down

```
docker build → Build an image
-t           → Assign a name/tag
my-first-image → Image name
.           → Current directory
```

The `.` means:

| Look for the Dockerfile in the current directory/context.

11. Docker Image Layers

One of the important concepts from the lecture is **Docker image layers**.

Suppose we have:

```
FROM ubuntu
CMD ["echo", "Hello Docker"]
```

The Ubuntu image provides the base layer.

However, `CMD` does **not** create a new filesystem layer. It modifies image metadata/configuration.

So if the Dockerfile only changes `CMD`, Docker may reuse the existing Ubuntu image layer.

12. `RUN` vs `CMD`

This is an important distinction.

`RUN`

```
RUN echo "Hello Docker"
```

`RUN` executes a command **while building the image**.

It creates a new image layer when it changes the filesystem.

`CMD`

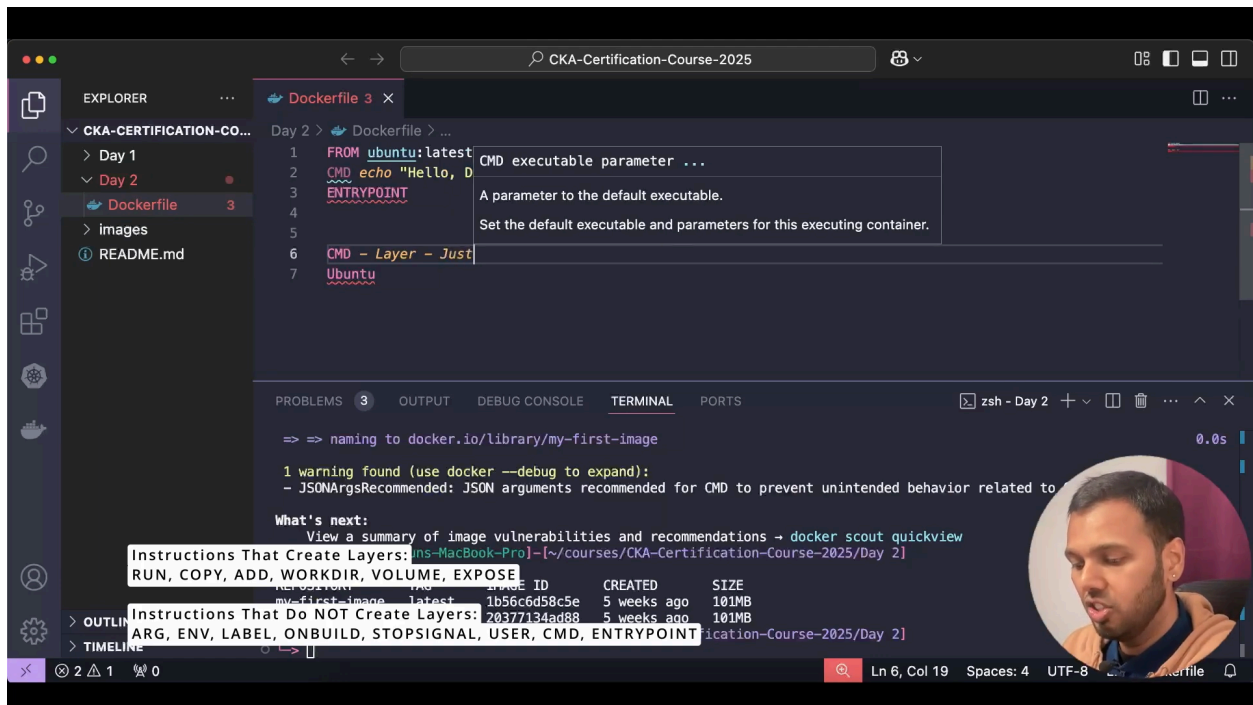
```
CMD ["echo", "Hello Docker"]
```

`CMD` specifies the command that should run **when the container starts**.

It does not create a normal filesystem layer in the same way `RUN` does.

Easy way to remember

```
RUN → Build time  
CMD → Container runtime
```



13. Example Dockerfile

```
FROM ubuntu
```

```
RUN echo "Creating an image"
```

```
CMD ["echo", "Hello Docker"]
```

Conceptually:

```

Ubuntu base image
↓
RUN instruction → new filesystem layer
↓
CMD metadata
↓
Final Docker image

```

14. **docker history**

To see how an image was constructed:

```
docker history my-first-image
```

It shows the image's layers/instructions.

Useful for understanding:

- Image layers
 - Commands used to create layers
 - Image history
-

15. **docker inspect**

To get detailed information about an image:

```
docker inspect my-first-image
```

This provides detailed metadata/configuration associated with the image.

16. Remove an Image

The command is:

```
docker rmi <image>
```

Example:

```
docker rmi my-second-image
```

To force removal:

```
docker rmi -f my-second-image
```

Where:

```
-rmi → remove image  
-f   → force
```

17. Run a Container

To create and run a container from an image:

```
docker run my-first-image
```

If the Dockerfile contains:

```
CMD ["echo", "Hello Docker"]
```

The output will be:

```
Hello Docker
```

18. Container Lifecycle

A very important Docker concept:

| A container generally runs only while its main process is running.

For example:

```
CMD ["echo", "Hello Docker"]
```

The container starts:

```
Container starts
  ↓
echo "Hello Docker"
  ↓
Command finishes
  ↓
Container exits
```

So seeing an **Exited** container after `docker run` is expected in this example.

19. `docker ps`

To list currently running containers:

```
docker ps
```

If nothing is running, you'll see no containers listed.

Why?

The container from the example already completed its command and exited.

20. `docker ps -a`

To list **all containers**, including stopped/exited containers:

```
docker ps -a
```

- `a` means:

```
all
```

This shows containers in states such as:

- Running
- Exited

- Stopped

Important exam point

```
docker ps
```

→ Running containers only.

```
docker ps -a
```

→ All containers.

21. Tagging an Image

Before pushing an image to Docker Hub, the image needs an appropriate repository name/tag.

Suppose your local image is:

```
my-first-image
```

You can tag it as:

```
docker tag my-first-image cloudwithvaros/my-first-image:v1
```

General syntax:

```
docker tag SOURCE_IMAGE TARGET_IMAGE
```

After tagging, Docker has another reference to the same image.

You can verify it:

```
docker images
```

22. Docker Image Tags

A tag identifies a particular version/reference of an image.

Example:

```
my-first-image:v1  
my-first-image:v2  
my-first-image:latest
```

Tags are commonly used to identify different versions.

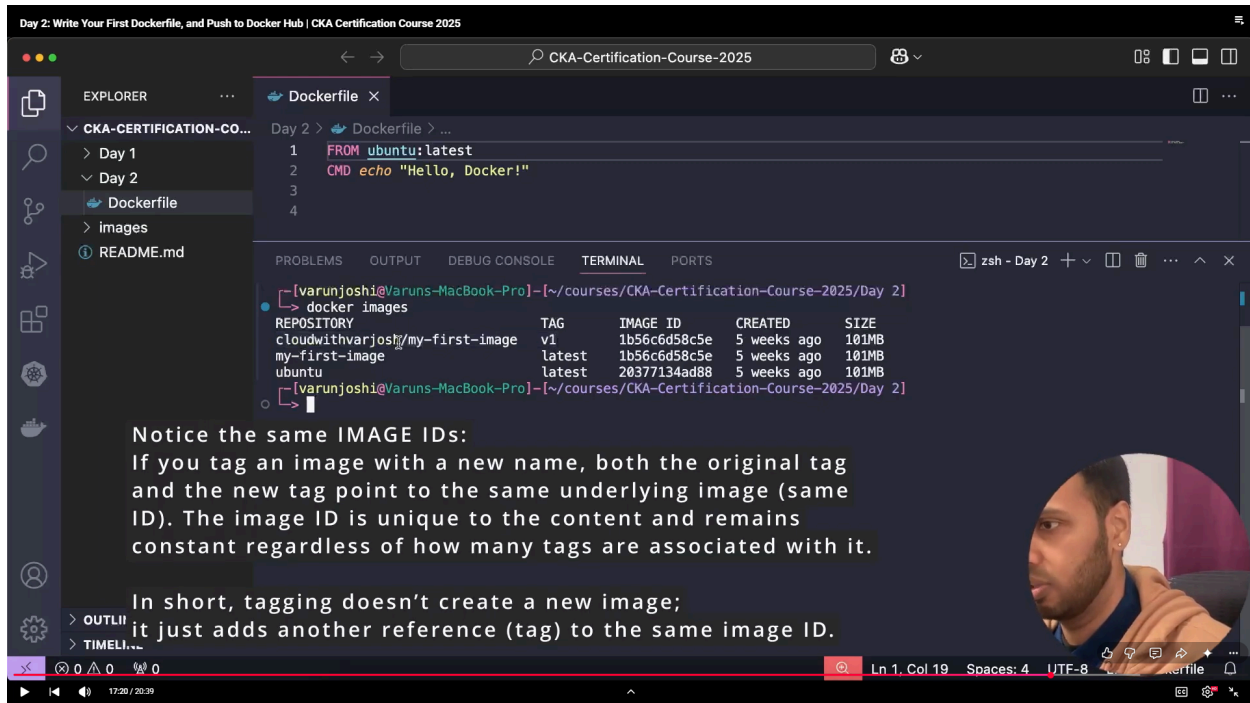
For example:

```
v1  
v2  
v3
```

Important

latest is a tag too.

It does **not** necessarily mean "newest image in existence"; it is simply the conventional default tag when no tag is specified.



23. Push Image to Docker Hub

After tagging the image correctly:

```
docker push cloudwithvaros/my-first-image:v1
```

Docker uploads the image to the corresponding Docker Hub repository.

The general format is:

```
docker push docker.io/<username>/<image>:<tag>
```

For example:

```
docker push docker.io/cloudwithvaros/my-first-image:v1
```

You can normally omit `docker.io` :

```
docker push cloudwithvaros/my-first-image:v1
```

Docker treats Docker Hub as the default registry in this context.

24. Why Can't You Just Push `my-first-image` ?

If you run:

```
docker push my-first-image
```

Docker doesn't know **which Docker Hub account/repository** should receive the image.

You need to tag it with your Docker Hub username:

```
docker tag my-first-image <username>/my-first-image:v1
```

Then:

```
docker push <username>/my-first-image:v1
```

25. Docker Authentication

If you're using Docker Desktop and you're already logged in, Docker can use that authentication context.

On a Linux/cloud VM, you may need to explicitly log in:

```
docker login --username <username>
```

Docker then prompts you for authentication.

After successful authentication, you can push images to your registry account.

For modern Docker workflows, using a personal access token instead of an account password is generally preferable when the registry supports it.

26. Docker Configuration

Docker stores authentication/configuration information in a configuration file under the user's Docker directory.

The lecture demonstrates inspecting it with:

```
cat ~/.docker/config.json
```

This can help you understand which Docker authentication/configuration context is being used.

Do not share credentials or authentication tokens contained in configuration files.

27. Docker Registry Examples

Docker Hub is not the only possible registry.

Other cloud/platform registries can have their own image naming conventions.

Conceptually:

```
Docker Hub  
docker.io/<username>/<image>:<tag>
```

Other registries may use a registry-specific URI such as:

```
<registry>/<project>/<image>:<tag>
```

The exact format depends on the registry.

28. `docker info`

To see detailed information about your Docker installation:

```
docker info
```

It provides information about things such as:

- Docker environment
- Containers
- Images
- Storage
- Runtime
- Server configuration

29. Important Docker Commands — Cheat Sheet

Command	Purpose
<code>docker --version</code>	Show Docker version
<code>docker version</code>	Detailed version information
<code>docker info</code>	Show Docker system information
<code>docker --help</code>	Show available commands
<code>docker pull ubuntu</code>	Download Ubuntu image
<code>docker images</code>	List local images
<code>docker build -t name .</code>	Build an image
<code>docker run image</code>	Create/run a container
<code>docker ps</code>	List running containers
<code>docker ps -a</code>	List all containers
<code>docker rmi image</code>	Remove an image

Command	Purpose
<code>docker rmi -f image</code>	Force-remove an image
<code>docker history image</code>	Show image history/layers
<code>docker inspect image</code>	Show detailed image information
<code>docker tag source target</code>	Create/tag an image reference
<code>docker push image:tag</code>	Push image to registry
<code>docker login</code>	Authenticate with registry

30. End-to-End Workflow

The overall workflow from this lecture can be remembered as:

```
Install Docker
  ↓
Verify Docker
  ↓
Login to Docker Hub
  ↓
Pull base image
  ↓
Create Dockerfile
  ↓
docker build
  ↓
Docker Image
  ↓
docker run
  ↓
Container
  ↓
docker tag
  ↓
docker push
  ↓
Docker Hub Repository
```

31. Key Concepts to Remember for CKA/Interview

Image vs Container

Image

- Read-only template/package
- Used to create containers
- Can be stored in a registry

Container

- Running/created instance of an image
- Has its own runtime environment
- Stops when its main process exits

```
Image
  ↓
docker run
  ↓
Container
```

Registry vs Repository vs Image

```
Registry
  ↓
Repository
  ↓
Image + Tag
```

Example:

```
Docker Hub
  ↓
ubuntu repository
  ↓
ubuntu:latest
ubuntu:24.04
```

RUN VS **CMD**

RUN
↓
Executed while building image
↓
Can create filesystem layer

CMD
↓
Default command when container starts
↓
Image metadata/configuration

docker ps VS **docker ps -a**

docker ps
↓
Running containers

docker ps -a
↓
All containers
↓
Running + stopped + exited

32. Most Important Commands to Practice

If you're practicing this lecture hands-on, these are the commands I'd memorize first:

```
docker --version
```

```
docker pull ubuntu
```

```
docker images

docker build -t my-first-image .

docker run my-first-image

docker ps

docker ps -a

docker history my-first-image

docker inspect my-first-image

docker tag my-first-image <username>/my-first-image:v1

docker push <username>/my-first-image:v1

docker info
```

One-line memory trick

Pull → Build → Run → Tag → Push

```
Registry
  ↓ pull
Image
  ↓ run
Container

Dockerfile
  ↓ build
Image
  ↓ tag
Registry name
  ↓ push
Docker Hub
```

This is the core flow of the entire lecture.